

第 2 章

MCPの アーキテクチャ

Model Context Protocol

2.1 MCPのアーキテクチャ

本節では、MCPアーキテクチャで、どのようなコンポーネントが登場するか、コンポーネント間でどのように通信するかといった概要を紹介します。詳細については次節以降で説明していきます。

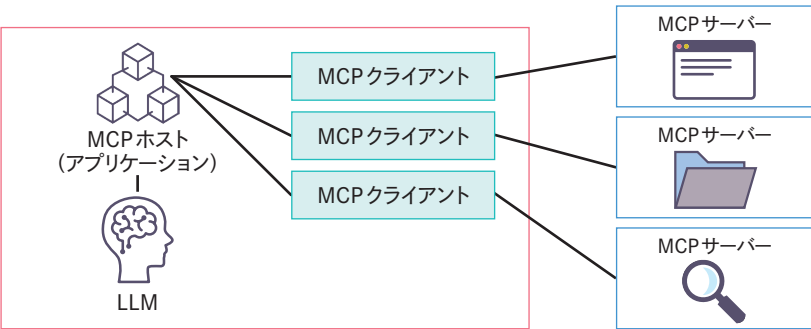
MCPはクライアントサーバーモデルを採用したアーキテクチャとなっています。コンテキストを提供するMCPサーバーと、MCPサーバーにコンテキストをリクエストするMCPクライアントが存在します。1つのMCPクライアントは1つのMCPサーバーと1対1に通信してコンテキストをやり取りします。

複数のMCPサーバーと通信するためには、MCPサーバーの数に応じたMCPクライアントが必要となります。MCPクライアントを生成し、管理する役割を持つのがMCPホストとなります。MCPではこの3つのコンポーネントで構成されます。

コンポーネント	概要
MCPサーバー	MCPクライアントにコンテキストを提供する
MCPクライアント	MCPサーバーとのセッションを作成し、コンテキストを取得する
MCPホスト	MCPクライアントを生成する MCPクライアントを権限やライフサイクルを管理する 取得したコンテキストを管理する

MCPでは、これら3つのコンポーネント間でコンテキストをやり取りします。インターネット上で、Webサイト検索、GitHubのリポジトリ操作、AWSリソースの操作などの機能を持ったMCPサーバーが公開され、広く活用されています。MCPホストは、AIコーディングエージェントやAIエージェントフレームワークで構築したLLMアプリケーションなどが該当します。MCPホスト内部でMCPクライアントを生成して、MCPサーバーと接続します。

MCPコンポーネントの概要図

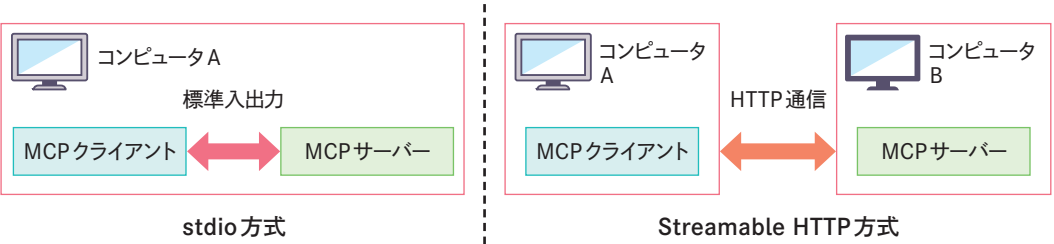


MCPコンポーネント間の通信は、データレイヤーとトランスポートレイヤーの2つのレイヤーで構成されています。概念的にはデータレイヤーが内側で、トランスポートレイヤーが外側となります。

データレイヤーでは、MCPサーバーとMCPクライアント間の通信に、JSON-RPC 2.0をメッセージ形式として用いるように規定されています。

トランスポートレイヤーでは、データを送受信する手段が定義されています。トランスポートとしてstdio方式とStreamable HTTP方式があります。stdio方式ではMCPサーバーとMCPクライアントが同一マシン上で標準入出力を通して通信します。Streamable HTTP方式では異なるマシンで動作するMCPサーバーとMCPクライアントがネットワークを介してHTTP通信します。

トランスポートレイヤーにおけるstdio方式とStreamable HTTP方式



MCPサーバーとMCPクライアントが互いに提供する機能をPrimitive（プリミティブ）と呼びます。MCPホストであるAIアプリケーションは、このプリミティブによって、どのようなコンテキスト情報が利用可能かを判断します。

提供元	プリミティブ	概要
MCPサーバー	Prompts (プロンプト)	再利用可能なプロンプトテンプレート
	Resources (リソース)	ファイルのテキストやデータベーススキーマなどのコンテキスト
	Tools (ツール)	データ書き込みや検索などのアクションを実施する実行可能関数
MCPクライアント	Sampling (サンプリング)	MCPクライアントがMCPサーバーに代わって、LLM推論を実施
	Roots (ルート)	MCPサーバーが操作可能なファイル階層のルートを指定
	Elicitation (エリシテーション)	MCPサーバーからユーザーに対する追加情報の要求を受け入れ

2.2 MCP通信のメッセージ形式

MCPサーバーとMCPクライアントにおける通信では、メッセージ形式として、JSON-RPC 2.0^{注2.1} プロトコルを用いることが規定されています。JSON-RPC 2.0は、リモートプロシージャコール (Remote Procedure Call : RPC) のためのプロトコルです。通信するデータ形式としてJSONを採用しており、ステートレスかつ軽量なことが特徴です。またPythonやTypeScriptなど多くのプログラミング言語で簡単に扱うためのライブラリが提供されています。可読性も高いため、デバッグも効率的にできます。

MCPの通信において、実際にどのようなJSON-RPCメッセージがやり取りされるのか見てみましょう。まずはMCPクライアントからMCPサーバーに送信されるリクエストメッセージのサンプルです。

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "echo",
    "arguments": {
      "message": "Hello World!"
    }
  }
}
```

注2.1 <https://www.jsonrpc.org/>

リクエストメッセージでは、呼び出すメソッドとその時のパラメータを指定します。このサンプルでは「echo」という名前のToolsプリミティブを、message引数に「Hello World!」という値を指定して呼び出しています。

続いてレスポンスメッセージのサンプルです。

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Echo: Hello from Claude Code!"
      }
    ]
  }
}
```

レスポンスメッセージでは、「result」で応答内容を返します。どのリクエストに対するレスポンスであるかを明確にするために「id」の値をリクエストの「id」の値と一致させる必要があります。

2.3 MCPの通信プロトコル

MCPでは通信方式として、stdio方式とStreamable HTTP方式の2つが定義されています。

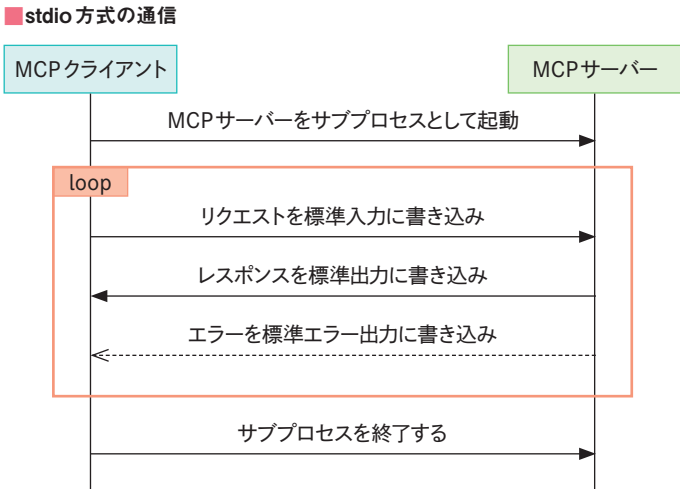
2.3.1 stdio方式

stdio方式は、MCPサーバーとMCPクライアントが同じマシン上で動作する際に利用されます。このようなMCPサーバーをローカルMCPサーバーとも呼びます。stdio方式の代表的な特徴は以下です。

- MCPクライアントがMCPサーバーをサブプロセスとして起動する
- メッセージのやり取りに、標準入出力を用いる
 - MCPクライアントは、メッセージをMCPサーバープロセスの標準入力に書き込む

- MCPサーバーは、メッセージを標準出力に書き込む
- MCPサーバーは、ログなどのUTF-8文字列を標準エラー出力に書き込んでもよい

stdio方式での通信シーケンスのイメージ^{注2.2}は以下となります。



2.3.2 ■ Streamable HTTP方式

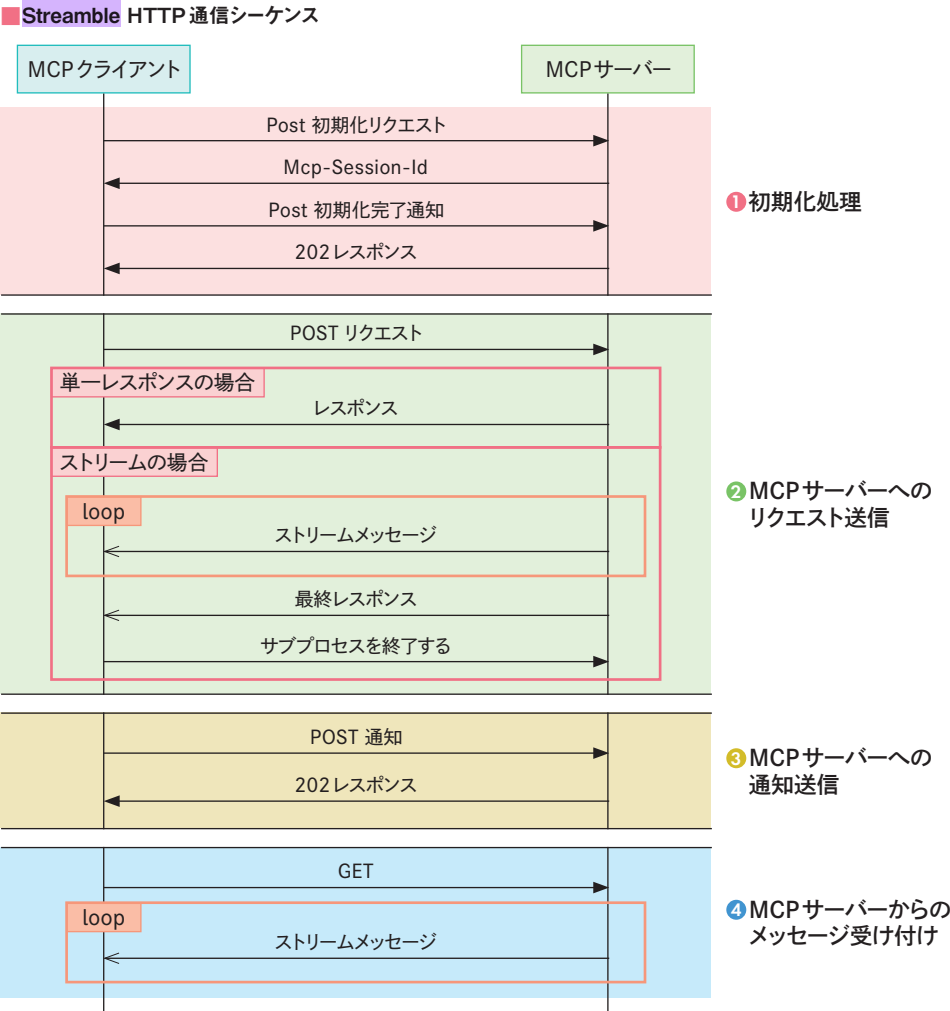
Streamable HTTP方式では、MCPサーバーとMCPクライアントが、異なるマシン上で動作し、ネットワークを介して通信します。この通信方式で動作するMCPサーバーをリモートMCPサーバーと呼びます。

MCPクライアントは、HTTP POSTメソッドでMCPサーバーとの通信を開始します。応答が単一メッセージとなるときは、MCPサーバーはJSON-RPC形式の応答を返します。ストリーミング通信が必要な時は、Server-sent events (SSE) による通信を開始し、このSSEストリームの中でメッセージをやり取りします。

MCPサーバーをクラウドなどでホスティングすることで、利用者は自身の環境を変更することなく、MCPサーバーが提供する機能を利用できます。

Streamable HTTP方式での通信シーケンスのイメージ^{注2.3}は以下となります。

注2.2 <https://modelcontextprotocol.io/specification/2025-06-18/basic/transport>を参考に作成
注2.3 <https://modelcontextprotocol.io/specification/2025-06-18/basic/transport>を参考に作成



「① 初期化処理」では、MCPクライアントとMCPサーバー間で接続セッションを確立します。MCPサーバーは、初期化リクエストに対してMcp-Session-Idを返すことができます。②～④の処理ではこのMcp-Session-IdをMCPクライアント側から指定することで、セッションを指定できます。

「② MCPサーバーへのリクエスト送信」では、MCPクライアントからリクエストを送信します。レスポンスがストリーム形式となる場合は、SSEストリームを通して、MCPサーバーはレスポンスを返します。

「③ MCPサーバーへの通知送信」はMCPクライアントから通知を送る場合のシーケンスです。

「④ MCPサーバーからのメッセージリッスン」では、MCPクライアントからMCPサーバーにGETリクエストを送ることで、MCPサーバーからのメッセージが受信可能になります。

①のシーケンスで初期化した後に②、③、④のシーケンスに従って通信をします。ToolsプリミティブのMCPツール実行などは②のシーケンスとなります。Rootsプリミティブでルート変更を通知するケースでは③のシーケンスとなります。④のシーケンスは、SamplingプリミティブやElicitationプリミティブでMCPサーバー側からのリクエストを受け付ける際に利用されます。

Column

SSE (Server-Sent Events) 方式

MCPが初めて登場したとき、通信方式としてはstdio方式とSSE方式がサポートされていました。SSEでは高可用性を維持したまま接続を安定的に保つ必要があるなど、いくつかの課題がありました。そのため、Version 2025-03-26からはStreamable HTTP方式へと置き換えられました。Streamable HTTP方式ではSSE方式の課題を解決し、Mcp-Session-Idヘッダの導入によって、セッション管理機能などが実現できるようになっています。

2.4

MCPプリミティブ

2.4.1

Promptsプリミティブ

プロンプトテンプレートを提供するプリミティブです。プロンプトテンプレートはLLMへの指示であるプロンプトの一部を変数としておき、利用時に具体的な値を入れることができます。ユースケースに応じて、プロンプトをカスタマイズでき、再利用性を高めることができます。

■プロンプトテンプレートの例

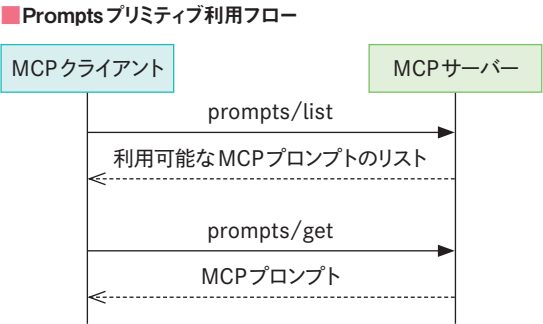
あなたは優秀なリサーチャーです。
{topic}に関する情報をWebから収集し、それを基にレポートを生成してください。

Promptsプリミティブでは、2つのプロトコルを用いて情報をやり取りします。

メソッド	提供機能
prompts/list	利用可能なMCPプロンプト一覧を取得する
prompts/get	特定のMCPプロンプトを取得する

MCPサーバーがどのようなプロンプトテンプレートを提供しているかは、prompts/listで取得できます。取得した情報には、プロンプトの目的やプロンプトテンプレートに埋め込む変数の情報が含まれます。

具体的なプロンプトを取得するにはprompts/getを使います。使いたいプロンプトテンプレート名称と必要な変数値をリクエストに含めることで、LLMが利用可能なプロンプトを取得できます。



2.4.2

Resourcesプリミティブ

LLMに対して、読み取り専用のコンテンツを提供するプリミティブです。コンテンツは社内マニュアルや手順書、JSONファイルなどのテキストファイルはもちろん、画像やPDFなどのバイナリファイルも受け渡し可能です。

リソースは一意的なURIをもち、これにより識別します。URIはスキーマ、ホスト、コンテンツのパスによって構成されます。

<スキーマ>://<ホスト>/<コンテンツのパス>

動的なリソースを定義するために、URIテンプレートを使ってパラメータ化されたリソーステンプレートを使用することもできます。

- リソーステンプレートの例
file:///documents/{date}/report.txt

Resources プリミティブで利用できるメソッドは4種類あります。

メソッド	提供機能
resources/list	利用可能なMCPリソース一覧を取得する
resources/template/list	利用可能なMCPリソーステンプレート一覧を取得する
resources/read	MCPリソースのコンテンツを取得する
resources/subscribe	MCPリソースの変更通知をサブスクライブする

2.4.3 Toolsプリミティブ

LLMに対して、外部環境と連携する機能を提供するプリミティブです。LLMは、事前学習した知識を用いた受け答えしかできませんが、ツールを活用することで、外部から情報を取得したり、ファイルを作成したりするなど、機能を拡張できます。

Toolsプリミティブでは、LLMに外部環境と連携する機能を提供するために、2種類のメソッドを定義しています。

メソッド	提供機能
tools/list	MCPサーバーが提供するMCPツールの一覧を取得する
tools/call	指定したMCPツールを実行する

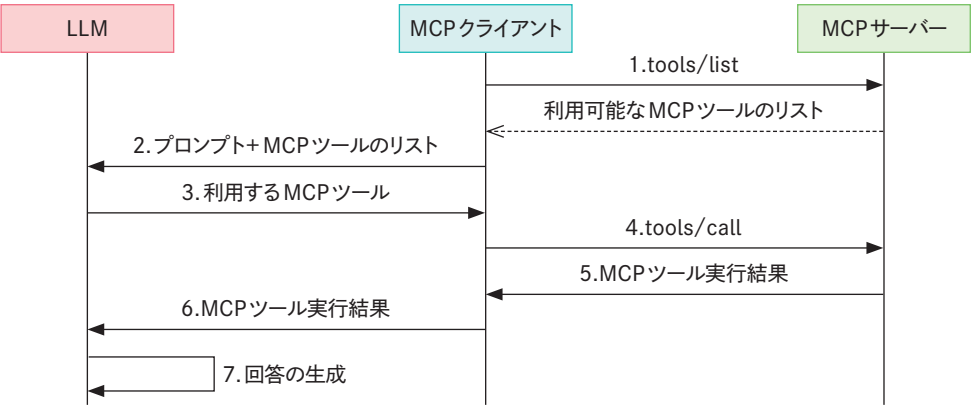
tools/listは、MCPサーバーが提供するツールの一覧を取得するメソッドです。MCPホストはMCPサーバーにtools/list リクエストを送ることで、利用可能なツール情報の一覧を取得できます。ツール情報には、ツールの概要や入力スキーマ (inputSchema) が含まれるため、これを元に、いつ、どのようにツールを使えばよいか判断できます。

tools/callは、ツールを実行するメソッドです。リクエストを送ることでMCPサーバー側で処理を実施し、その結果を返します。

Toolsプリミティブの利用フローは以下のとおりです。

- 1. MCPクライアントは、tools/listメソッドで利用可能なMCPツール一覧を取得する
- 2. MCPクライアントは、プロンプトとMCPツール一覧をLLMに渡す
- 3. LLMは、どのMCPツールを利用するか決定する
- 4. MCPクライアントは、tools/callメソッドでMCPサーバーにツール実行をリクエストする
- 5. MCPサーバーは、リクエストがあったツールを実行し、結果を返す
- 6. MCPクライアントは、ツール実行結果をLLMに渡す
- 7. LLMは、ツール実行結果をもとに最終回答を生成する

Toolsプリミティブ利用フロー

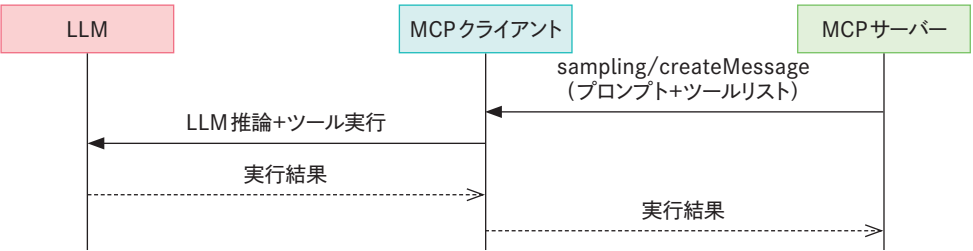


2.4.4 Samplingプリミティブ

MCPサーバーがLLMを呼び出す必要があるときに、その呼び出しをMCPクライアントに依頼するプリミティブです。Samplingプリミティブを使うことで、LLM呼び出しやツール実行をMCPクライアントに集約でき、LLMの利用コストや必要な認証情報を一元管理できます。

MCPサーバーからMCPクライアントへは、sampling/createMessage リクエストによりLLM呼び出しを依頼します。このリクエストには、プロンプトや必要なパラメータの情報に加え、使用すべきツールリストも含まれます。MCPクライアントはこの情報を基に、LLMを呼び出し、その結果をMCPサーバー側に伝えます。

Samplingプリミティブの想定フロー



MCPサーバーからsampling/createMessage リクエストを受け取ったときに、無条件で実行すると、予期せぬプロンプトの利用やツール実行の可能性があります。セキュリティを担保するためにも、受信したリクエストは人が介在して精査するとよいでしょう。

2.4.5 ■ Rootsプリミティブ

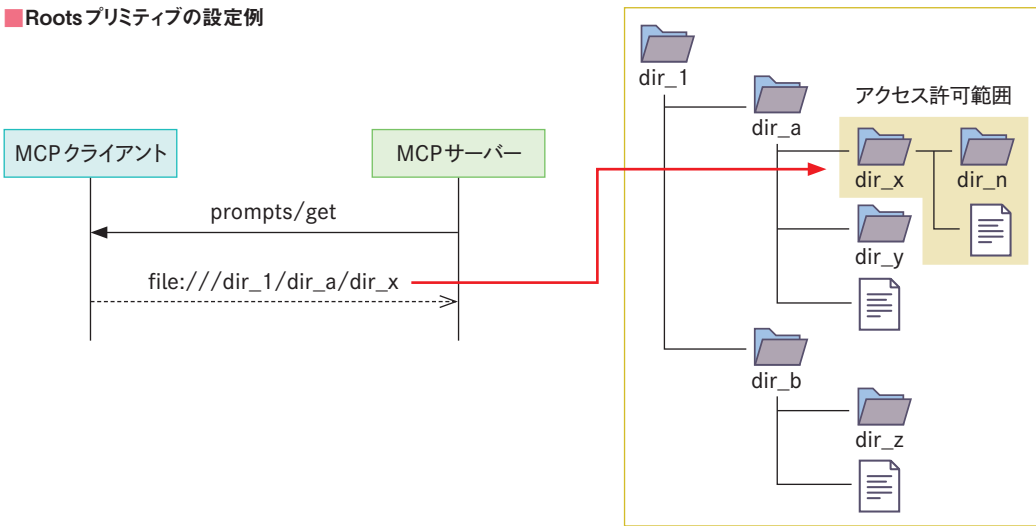
RootsプリミティブはMCPクライアントが提供するプリミティブで、MCPサーバーに操作を許可する範囲を提供します。

ファイルを読み書きするツールをMCPサーバーが提供する場合、そのアクセス制御が重要となります。MCPサーバーが、すべてのファイルを自由に読み書きできてしまうと、機密情報が流出したり、予期せぬファイル更新が発生したりするリスクがあります。

このリスクを軽減するために利用されるのがRootsプリミティブです。MCPサーバーはroots/listリクエストを送ることで、“file:///”で指定されたパスを受け取ることができます。

Rootsプリミティブで指定されたアクセス許可範囲を遵守するかどうかはMCPサーバーの実装に依存します。システムレベルでアクセス許可範囲外へのアクセスを完全に防ぐことはできないので注意が必要です。

Rootsプリミティブの設定例



2.4.6 ■ Elicitationプリミティブ

MCPサーバーがユーザーに対して、必要な情報を必要なタイミングで確認するためのプリミティブです。Elicitationプリミティブでは、確認するデータの種類やフローに応じて2つのモードをサポートしています。

Elicitationプリミティブのモード

モード	概要
Form	構造化データをやり取りし、ユーザーから必要な情報を収集する
	ユーザーが入力した情報はMCPクライアントを経由して、MCPサーバーに通知する
URL	MCPサーバーが指定したURLにユーザーを遷移させ、認証情報など必要な情報を入力させる
	URL以外の情報は、MCPクライアントを経由しない

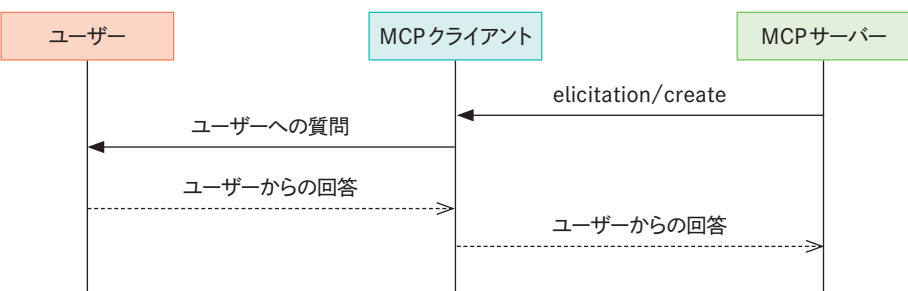
MCPサーバーはelicitation/createリクエストを通して、モードやメッセージなどをMCPクライアントに送ります。Formモードの場合、MCPクライアントはユーザーに情報入力を促すUIを表示します。URLモードの場合は指定されたURLにユーザーを誘導します。

ユーザーのレスポンスを明確に区別するために、3つのレスポンスアクションが定義されています。MCPサーバーはレスポンスアクションに応じた処理が必要となります。

レスポンスアクション

レスポンスアクション	概要	MCPサーバーでの処理
Accept	ユーザーが明示的に承認してデータを提示した	提示されたデータを処理する
Decline	ユーザーが明示的にリクエストを拒否した	代替案の提示など、ユーザーの拒否に対応する
Cancel	ユーザーが明示的な判断をせず、終了した	再提示など、キャンセル処理に対応する

Elicitationプリミティブの想定フロー（Formモード）



2.5 MCPにおける認証・認可

2.5.1 認証・認可の必要性

MCPサーバーが提供する機能の中には、MCPサーバーがデータベースにアクセスする機能を持つ場合があります。MCPサーバーがどのようなデータにアクセスできるかは利用ユーザーが持つ権限によって異なります。アクセス範囲を決定するためには、ユーザーを認証し、ユーザーが許可されたアクセス範囲を決定する認可が必要となります。

特に Streamable HTTP 方式を用いたリモート MCP サーバーでは、認証情報やアクセス保護されたデータを適切な方法で扱わないと、情報漏洩のリスクは高まります。MCP では、このリスクを低減するために、MCP サーバーと MCP クライアントとの間での認証・認可フローを定義しています。

本書では認証・認可に焦点を当てたハンズオンはありませんが、アップデートによる仕様変更が大きく、重要なテーマであるため、コアとなる技術要素とフローについて紹介します。

2.5.2 認証・認可を支える技術「OAuth」

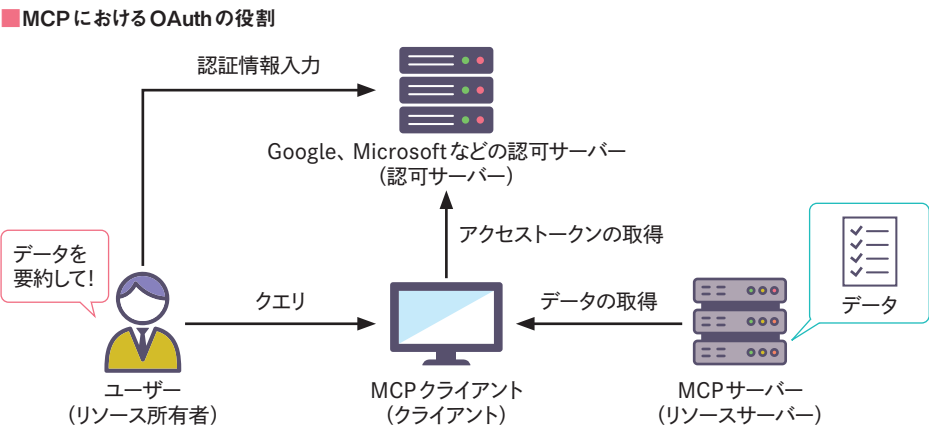
保護されたリソースを扱う MCP サーバーを利用するには、利用者が持つ権限を MCP サーバーに提示する必要があります。提示するために必要な認証情報を正しく扱わないとセキュリティリスクとなるため、安全な方法でやりとりする必要があります。Web サービスの分野では OAuth という認可の標準規格が広く採用されています。MCP の通信においても同様に、OAuth をベースとしたフローが採用されています。

OAuth は、クライアントアプリケーションが、コンテンツの所有者に代わって、リソースを管理するサーバーに安全にアクセスするための手順を定義しています。MCP では、MCP サーバーと MCP クライアント間での認証と認可にこの OAuth の仕組みを利用しています。

OAuth では、登場人物として次の4つが定義されています。

登場人物	役割
リソース所有者	リソースの所有者 クライアントへリソースアクセスを許可するエンドユーザー
リソースサーバー	リソースを管理するサーバー クライアントから受け取ったトークンを使い、リソースへのアクセスリクエストを処理する
クライアント	リソース所有者の代わりに、リソースサーバーにリソースへのアクセスをリクエストする
認可サーバー	リソース所有者を認証し、アクセストークンをクライアントに発行するサーバー

これを MCP のコンポーネントに当てはめると下図のようになります。



OAuthでは基本的な OAuth 2.1 認可フレームワークのほかに拡張機能を定義した多くの規格が存在します。その中で MCP サーバー、MCP クライアント、認可サーバーで実装が必須、推奨、または任意とされているものは以下となります^{注2.4}。

注2.4 Version 2025-11-25 ベース

標準規格	概要	対象	要求レベル
OAuth 2.1 IETF ドラフト (draft-ietf-oauth-v2 ^{注2.5)}	クライアントがアクセストークンを受け取り、リソースサーバーの持つリソースを安全に取得するためのフローを定義	認可サーバー、MCPクライアント	必須 (MUST)
OAuth 2.0 認可サーバーメタデータ (RFC8414 ^{注2.6)}	認可サーバーのメタデータを取得する手順とメタデータ形式を定義	認可サーバー、MCPクライアント	必須 (MUST) ^{注2.7}
OAuth 2.0 動的クライアント登録 (RFC7591 ^{注2.8)}	クライアントが認可サーバーに、動的に登録するための手順を定義	MCPクライアント、認可サーバー	任意 (MAY)
OAuth 2.0 保護リソースメタデータ (RFC9728 ^{注2.9)}	リソースサーバーのメタデータを取得する手順とメタデータ形式を定義	MCPサーバー、MCPクライアント	必須 (MUST)
OAuth クライアント ID メタデータ ドキュメント (draft-ietf-oauth-client-id-metadata-document-00 ^{注2.10)}	HTTPS URLをクライアントIDとして使用することで、認可サーバーへの事前登録を不要とする手順を定義	MCPクライアント、認可サーバー	推奨 (SHOULD)
OAuth 2.0 リソースインジケータ (RFC8707 ^{注2.11)}	クライアントが認可サーバーにリクエストするアクセストークンの使用対象となるリソースサーバーを明示的に指定する手順を定義。アクセストークンの不正利用を防止する	MCPクライアント	必須 (MUST)

2.5.3 ■ 認証・認可フロー

MCPに定義される認証・認可フローを紹介します。本項では各手順の大まかなイメージを説明していきます。詳細なシーケンスは公式ドキュメント^{注2.12}にも記載があるため、そちらと合わせて確認すると、より理解が進むでしょう。

MCPにおける認証・認可フローのイメージは以下となります。

■ MCPにおける認証・認可フロー



注2.5 <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-v2-1-13>
注2.6 <https://datatracker.ietf.org/doc/html/rfc8414>
注2.7 OAuth 2.0 認可サーバーメタデータが OpenID Connect Discovery 1.0 の少なくともいずれかには対応が必須です。
注2.8 <https://datatracker.ietf.org/doc/html/rfc7591>
注2.9 <https://datatracker.ietf.org/doc/html/rfc9728>
注2.10 <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-client-id-metadata-document-00>
注2.11 <https://datatracker.ietf.org/doc/html/rfc8707>
注2.12 [ここに最新版を](#)

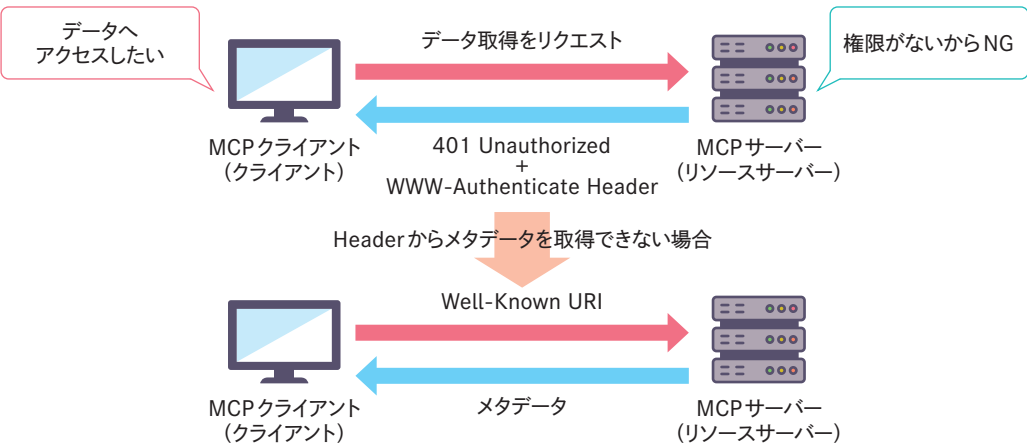
①アクセストークンなしにMCPサーバーへリクエスト

はじめに、MCPクライアントはMCPサーバーにリクエストを送ります。ユーザー認証は未実施のため、リクエストにアクセストークンは含まれていません。

MCPサーバーは、受け取ったリクエストにアクセストークンが含まれていないため、リクエスト元がデータアクセスに必要な権限を持っていないと判断します。クライアントへは、未承認を示すHTTPステータスコード「401 Unauthorized」を返します。そのときMCPサーバーは **WWW-Authenticate** のHTTPヘッダーにリソースサーバーである自身のメタデータURL情報を付与して返答します。MCPクライアントはHTTPヘッダーを確認してメタデータを取得します。仮にメタデータが含まれていない場合は、RFC 9728 で定義された **Well-Known URI** からメタデータを取得します。

MCPサーバーはWWW-AuthenticateのヘッダーかWell-Known URIのいずれかでメタデータを提供することが必須となります。MCPクライアントはどちらのケースでもメタデータを取得できるように実装することが必須と定義されています。

■ アクセストークンなしに MCPサーバーへリクエスト



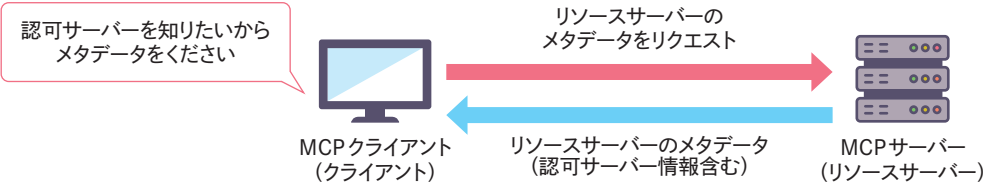
②MCPサーバーのメタデータを取得

MCPクライアントは、MCPサーバーから受け取ったメタデータURLを元に、必要なメタデータ取得をMCPサーバーにリクエストします。

MCPサーバーはリクエストに応じて、自身のメタデータを返します。メタデータには、**MCPサーバーが利用する認可サーバーの所在情報**が含まれます。MCPクライアントはこの情報に基づいて、どの認可サーバーに問い合わせればよいか確認できます。この仕組みは、OAuth 2.0保護リソースメタデータ (RFC 9728) に基づいており、MCPサーバーはこれに準拠することが必須要件と

して定義されています。

■ MCPサーバーのメタデータを取得

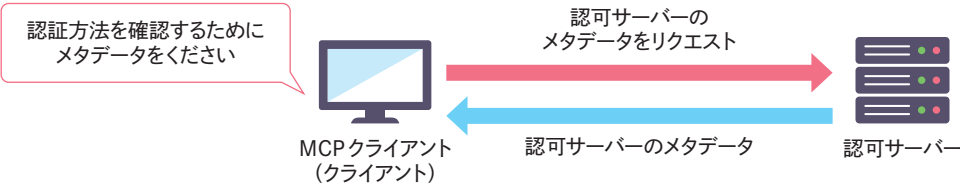


③ 認可サーバーのメタデータを取得

MCPクライアントは、②で取得した認可サーバーの所在情報を元に、認証方法の詳細を認可サーバーに問い合わせます。認可サーバーはリクエストに応じて自身のメタデータを返します。認可サーバーのメタデータには、自身とどのように対話すればよいかの情報が含まれています。

MCPクライアントは、OAuth 2.0 認可サーバーメタデータ (RFC 8414) と OpenID Connect Discovery 1.0の両方に対応するために、それぞれで定義されたエンドポイントを順番に呼び出す必要があります。

■ 認可サーバーのメタデータを取得



④ 認可サーバーへのMCPクライアント登録

これまでの処理でMCPクライアントは、どの認可サーバーに対して、どのように対話すればよいか知ることができました。次にユーザー認証し、アクセストークンを取得する必要があります。そのために、MCPクライアント自身を「認可サーバーのクライアント」として登録する必要があります。MCPではクライアント登録の方法として3つがサポートされています。

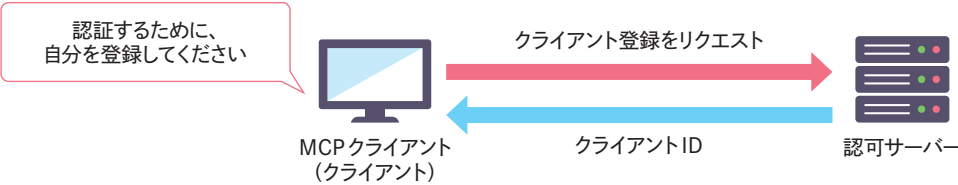
- 1. 事前登録
- 2. 動的クライアント登録
- 3. クライアントIDメタデータドキュメント

「事前登録」は、認可サーバーにMCPクライアントを登録しておき、クライアントIDを事前発

行する方法です。MCPクライアントは、発行されたクライアントIDを用いて認可リクエストを出します。

OAuth 2.0 動的クライアント登録 (RFC 7591) では、クライアントからの認可サーバーへの登録リクエストをトリガーに、認可サーバーでのクライアント登録を自動化します。これによって、MCPクライアントを認可サーバーに事前登録する手間を省くことができます。

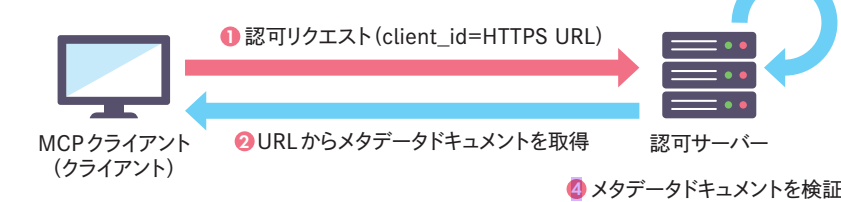
■ 動的クライアント登録



この方法では「https」スキームを持ったURLをクライアントIDとして利用することで、認可サーバーへの事前登録なしに認可フローを実現します。

MCPクライアントは自身のメタデータドキュメントをHTTPS URLで公開し、そのURLをクライアントIDとして使用します。認可サーバーは、クライアントIDとしてURLを受け取った場合、そのURLからメタデータドキュメントを取得して正当性を検証します。検証できた場合はクライアントの名称などを認可画面に表示し、ユーザー承認を受け付けます。

■ クライアントIDメタデータドキュメントの取得と検証



先ほど紹介した動的クライアント登録と同様に、認可サーバーへの事前登録なしにやり取りすることが可能です。動的クライアント登録で発生する登録クライアント管理や非アクティブなクライアントIDのクリーンアップなどの課題を解決するために、MCPではこの方法が推奨されています。

⑤ PKCE認証とアクセストークンの取得

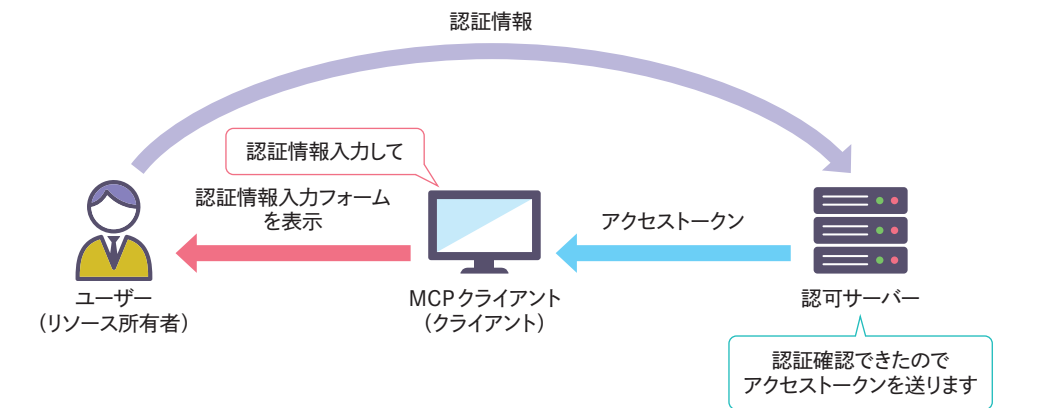
ユーザー認証を実施し、アクセストークンを取得するステップです。MCPクライアントを起点に、ユーザーが入力した認証情報を認可サーバーが検証し、結果としてアクセストークンをMCPク

クライアントに発行します。この認証フローではMCPクライアントと認可サーバー間で「認可コード」という一時トークンをやり取りします。

この認可コードを横取りして、不正にアクセストークンを取得する攻撃が存在します。MCPではこの横取り攻撃を避けるために、PKCEと呼ばれる手法の実装を必須としたOAuth 2.1の認可フローを適用することが求められています。

クライアントが認可サーバーに対して認可リクエストとアクセストークンをリクエストするときは、「どのMCPサーバーでトークンを利用するか」を明示する必要があります。これはリソースインジケータと呼ばれ、OAuthの拡張機能としてRFC 8707で規定されています。

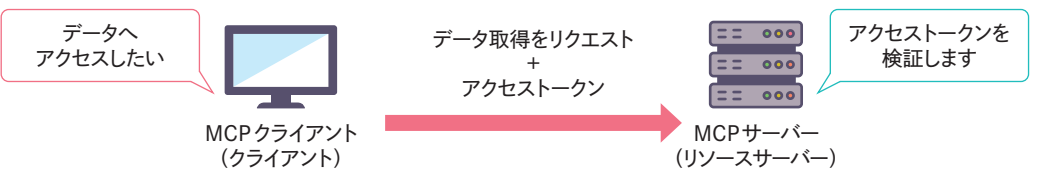
PKCE 認証とアクセストークンの取得



6 アクセストークンありでMCPサーバーにリクエスト

MCPクライアントは、取得したアクセストークンを用いて再度MCPサーバーにデータ取得をリクエストします。MCPサーバーではアクセストークンを検証し、問題がない場合にMCPクライアントのリクエストを処理します。

アクセストークンありでMCPサーバーにリクエスト



機密情報や個人情報を扱うMCPサーバーは、MCPクライアントが持つ権限に応じて適切に処理する必要があります。MCPではOAuthをベースとした認証フローを、双方が遵守することで、安全な通信を実現できます。

本節ではMCPの認証・認可フロー概要を紹介しました。紹介した内容以外にもスコープ管理やアクセストークンの利用方法など重要な要素が規定されているので最新の仕様を確認するようにしてください。

MCP仕様のバージョン

MCPの仕様は2024-11-04の初回リリースから、2025年12月時点の最新バージョンである2025-11-25まで改定が重ねられています。GitHubなどで、コミュニティの要望を基に仕様のディスカッションもされており、その提案も多く反映されています。今後も改定される予定のため、変更点などをチェックするとよいでしょう。

リリースバージョン	概要と大きな変化点
2024-11-04	※初回リリース ・プリミティブは5つ ・ Prompts ・ Resources ・ Tools ・ Sampling ・ Roots ・ 通信プロトコルはstdio方式とSSE方式
2025-03-26	・ OAuthベースの認証フローを導入 ・ SSE通信方式をStreamable HTTP通信方式に置き換え
2025-06-18	・ Elicitationプリミティブの導入 ・ Toolsプリミティブの構造化出力対応 ・ OAuthの認証フローでMCPサーバーをリソースサーバーとして定義
2025-11-25	・ Samplingプリミティブにツール利用を追加 ・ ElicitationプリミティブにURLモードを追加 ・ 認証フローでのOpenID Connect Discovery 1.0導入